

From First Grid to Reproducible Study

Project UAV Dynamic Routing	Research team Simra Imran and Muzzammil Sajid
Evidence cutoff 2026-07-27 22:03 PKT	Repository state Git f57ae15 + dirty expanded-study tree

Contents

The documentary is organized as a research narrative rather than a file changelog. Appendix-like technical detail, chronology, reproducibility, and glossary material appear after the remaining-work analysis.

How to read this documentary	5
Executive overview	6
The problem that motivated the research	7
Dynamic routing rather than static path finding	7
The original hypothesis	7
Why the comparison matters	8
The project at its beginning	8
Planned structure and collaboration	8
The Week 1 environment contract	8
Early learning approaches	9
The current routing world	9
Grid, graph, and coordinates	9
Static obstacles and exact scenario injection	10
Three dynamic-obstacle families	10
The information-timing contract	10
No-fly zones, energy, and sensor noise	11
What the learning agent sees and does	11
Goal-conditioned observation	11
Action and termination	11
Reward	11
Why the potential changed	12
Algorithms considered and implemented	13
Dijkstra	13
A*	13
D* Lite	13
Tabular Q-learning	13
DQN and Double DQN	13
Hindsight Experience Replay	13
How the research evolved	14
16 July: structure, grid, and Dijkstra	14
18-19 July: deep RL diagnosis and stabilization	14
21-22 July: dynamic obstacles and paired evaluation	15
23-25 July: paper construction and stronger baseline	15
26-27 July: research redesign, corrected training, and cloud migration	15
Problems discovered, fixes made, and lessons learned	16
Equal seeds were not equal environments	16
Dynamic obstacles could silently drift	16

Changes occurred at an ambiguous time	16
Action and observation changes invalidated models	16
HER terminal flags were wrong after relabeling	17
Reward design encouraged pathological value behavior	17
Obstacle-aware HER shaping was computationally and semantically unsafe	17
The first dynamic comparison was fair only by simplifying the world	17
One policy seed did not measure training uncertainty	17
Dijkstra alone was an incomplete classical comparison	17
Millisecond timings needed a protocol	18
Long training threatened reproducibility and local usability	18
The expanded research design	18
Research questions	18
The 310-scenario benchmark	18
Training matrix	19
Hyperparameters	21
Fair timing	21
Statistical plan	21
Adaptability as an event-level outcome	21
Integrity before interpretation	22
The computational workflow	22
Local development and training	22
Provenance lock	22
Why training moved to the cloud	23
Drive-backed resumability	23
What Drive backup does and does not guarantee	24
Results available at the cutoff	25
Historical static comparison	25
Historical Week 3 dynamic comparison	25
Historical unseen-layout diagnostic	26
Verified repeated classical pilot	26
Verified training completion	27
Verified implementation checks	27
Exact interim state	28
What remains before the research is finished	29
1. Finish the ablation training matrix	29
2. Reconcile provenance and stage coverage	29
3. Run the complete classical suite	29
4. Evaluate every learned variant	30
5. Pass the exact Cartesian integrity gate	30
6. Generate seed-aware summaries and formal tests	30
7. Generate figures, tables, and evidence-bound prose	30
8. Compile and visually inspect the release paper	30
9. Produce the definitive documentary	30
What the final results may establish	31

Expected contribution, conditional on final evidence	31
Reproducing and auditing the project	32
Key entry points	32
One-command pipeline order	32
Recreating the definitive documentary	33
Research limitations that remain even after completion	33
Chronological ledger	34
Technical appendix	34
DQN target	34
Path cost	34
Success and timeout	35
Confidence intervals and independence	35
Why local and full observation answer different questions	35
Glossary	35
Closing perspective	36

How to read this documentary

This is the interim documentary edition of the UAV Dynamic Routing research project. It reconstructs the project from its first committed structure on 2026-07-16 through the evidence available at 2026-07-27 22:03 PKT (UTC+05:00). It is not the final research paper, and it does not pretend that unfinished cloud experiments have already produced results.

The exact repository represented is `D:\UAV Dynamic Routing\uav-dynamic-routing`. The Git commit at the evidence cutoff was `f57ae150068fbd7a00c1b37a8280cf60a5dd2ddc`. The working tree also contained 63 modified or untracked status entries implementing the expanded study, so the commit hash alone is not a complete description of the state.

Four labels are used throughout:

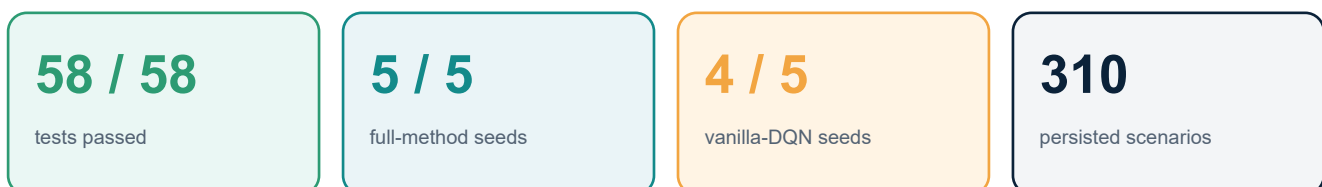
Label	Meaning in this document
VERIFIED	A completed artifact, test, result, checkpoint, or metadata record was inspected and passed the applicable check.
IN PROGRESS	Work had started and had a live status or partial checkpoint, but the final admissible artifact did not yet exist.
PLANNED	Code or a protocol may exist, but the required experiment or final generated evidence had not been completed.
INTERPRETATION	A reasoned explanation, hypothesis, or expected implication that must not be confused with a measured result.

The principal evidence is indexed as E01-E43 in the companion evidence manifest. File references in this documentary are project-relative unless an absolute path is shown. Early measurements are retained because they explain how the research evolved, but they are marked historical and are not merged with the corrected expanded-study results.

INTERIM EDITION. The expanded ablation campaign and final 310-scenario evaluation were incomplete at the cutoff. The definitive edition must replace pending claims with integrity-gated evidence.

Evidence status at the documentary cutoff

Implementation is further advanced than the final empirical evidence.



INTERIM - cloud training active

Active recorded task: `dqn:seed055` on free Google Colab T4

Final 310-scenario RL evaluation, statistics, integrity report, and release paper: pending



Figure 1. Status labels reflect evidence inspected at 2026-07-27 22:03 PKT; they do not infer completion from partial checkpoints.

Executive overview

The project began with a simple and appealing question: can a learned routing policy respond to a changing UAV environment more effectively than repeatedly running Dijkstra's shortest-path algorithm? The intuition was that Dijkstra offers a reliable optimal route for the graph it currently sees, but must search again when the graph changes. A trained reinforcement-learning policy, by contrast, can choose an action with a neural-network forward pass and may therefore become attractive when changes are frequent or maps are large.

The first design used a 15x15 two-dimensional grid, eight movement directions, seeded obstacles, straight movement cost 1, and diagonal movement cost square root of 2. The research paired a classical graph-search implementation with a reinforcement-learning environment. Static experiments were followed by fixed-position dynamic obstacles and curriculum-trained DQN+HER policies. The early paper reported an honest result: on its small controlled benchmark, Dijkstra was more reliable and produced shorter routes, while the learned policy completed most scenarios but did not demonstrate a statistically significant route-level compute advantage.

That early paper was useful, but it exposed a deeper methodological problem. One trained network, one classical baseline, a small grid, a simplified dynamic environment, and timing measurements without a full distribution could not support a broad claim about reinforcement learning versus classical planning. Several implementation issues also had to be corrected: independent random-number generators could produce different grids; dynamic obstacles had to be shared exactly; action and observation changes invalidated old checkpoints; HER relabeling needed corrected terminal flags; a reward design could inflate Q-values; an obstacle-aware shaping potential was not valid when recomputed under a different dynamic state; and the timing of environmental changes had to be identical for every method.

The study was therefore redesigned. The current research program compares Dijkstra, A*, D* Lite, and a five-seed Double DQN+HER method. It includes six controlled RL ablations, a persisted 310-scenario manifest, distribution-shift tests, scales from 15x15 to 100x100, stochastic and moving obstacles, energy penalties, no-fly variants, sensor noise, repeated timings, event-level adaptability metrics, formal paired statistical tests, source provenance, and an integrity-gated manuscript pipeline.

At the cutoff, the implementation was substantially ahead of the final evidence. All 58 local tests passed. All five corrected full-method seeds had completed 500,000 training steps each under the same locked source digest. Four of five vanilla-DQN ablation seeds were also complete. The remaining training had been migrated from the local Windows machine to a free Google Colab T4 runtime, with Google Drive used for seed-by-seed recovery and backup. The cloud status named [dqn:seed055](#) as the active task and recorded the local machine as stopped.

The strongest completed current result is a repeated classical pilot, not the final multi-method conclusion. Across 50 dynamic scenarios and ten timing repetitions per scenario, both Dijkstra and A* succeeded in every run and returned the same mean path cost. A* used far fewer node expansions and was faster in this implementation. This establishes that the stronger classical comparison matters; it does not yet tell us how the final RL policies compare across the expanded benchmark.

The research is therefore best understood as a transition from a promising course-project comparison into a reproducible experimental study. Its central achievement so far is not a headline victory for one algorithm. It is the construction of a fairer question, a controlled test bed, and an evidence pipeline capable of producing an answer that can be audited.

The research question is a multi-objective trade-off

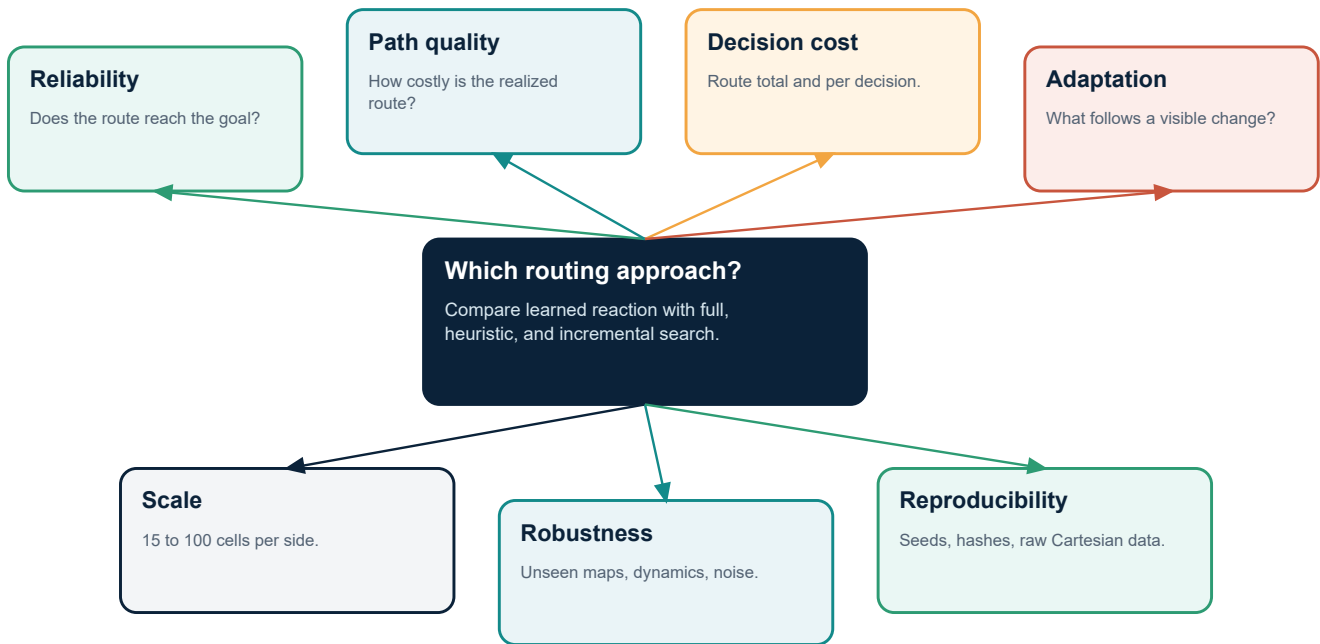


Figure 2. The expanded study refuses to reduce routing quality to a single speed or success number.

The problem that motivated the research

Dynamic routing rather than static path finding

A static path-planning problem asks for a route through a known map. A dynamic routing problem asks the vehicle to continue making safe and effective decisions when the map changes during execution. In a UAV setting, a route can be affected by moving vehicles, temporary restrictions, weather, local energy costs, sensor error, or a newly blocked corridor. Even if a planner found the best route at takeoff, that route can later become invalid or unnecessarily expensive.

The project deliberately narrows this broad operational problem into a controlled grid benchmark. A grid cannot represent full aerodynamics, three-dimensional motion, localization uncertainty, communications, or multi-UAV coordination. It does, however, make important algorithmic questions measurable:

- Is a route found?
- How expensive is the route under the shared movement-cost model?
- How much computation is used to make route decisions?
- What happens immediately after an environmental change?
- How consistent is a learned policy across independent training seeds?
- How do results change when the test distribution differs from training?

This abstraction is valuable only if its limits are stated. The project tests routing algorithms under controlled graph changes, not complete autonomous flight control.

The original hypothesis

The original project plan framed Dijkstra as the static optimal reference and reinforcement learning as an adaptive alternative. The motivating hypothesis was not that learning automatically produces shorter routes. It was that a trained policy might avoid repeated full graph search and therefore offer a deployment-time advantage as the environment becomes more dynamic or larger.

That hypothesis has two sides:

1. A computational advantage is useful only if success and path quality remain acceptable.
2. A result on one small map does not establish how either method scales.

The early experiments addressed the first side imperfectly and barely tested the second. The expanded study was designed to test both.

Why the comparison matters

Classical planners and learned policies fail in different ways. Dijkstra and A* calculate a shortest route on the graph they are given. Their route can become stale, but their search result is interpretable. D* Lite is designed to reuse search information when graph costs change. A learned policy has no shortest-path guarantee, can be sensitive to training seed and reward design, and can behave unexpectedly outside its training distribution. In exchange, the policy does not explicitly solve a graph-search problem at every decision.

The meaningful research question is therefore a trade-off question:

$$\text{deployment value} = f(\text{success, route cost, decision cost, adaptation, scale, robustness})$$

No single metric answers it. A policy that is fast but unsafe is not a useful winner; a planner that is optimal but prohibitively expensive at the target scale may also be unattractive. The expanded experiment treats these outcomes jointly.

The project at its beginning

Planned structure and collaboration

The original four-week plan divided the work between a data-structures and algorithms track and a reinforcement-learning track. The classical side built the grid representation, Dijkstra search, dynamic replanning, complexity analysis, and comparison metrics. The learning side built the Gymnasium environment, state/action/reward representation, DQN training, and dynamic fine-tuning. Both sides were supposed to meet at one shared environment contract and one evaluation table.

The plan started with 15x15 or 20x20 grids, a local sensor window, eight discrete actions, static obstacles, and later dynamic changes. It anticipated DQN or a Stable-Baselines3 implementation, a Dijkstra baseline, and metrics for path length, execution time, success rate, and adaptability. A curriculum from static to dynamic routing was part of the intended progression.

This division created a productive tension. Independent implementations made parallel work possible, but they also made silent environmental mismatch a serious risk. Much of the later research improvement work can be understood as turning an informal agreement into an executable shared contract.

The Week 1 environment contract

The earliest shared contract established:

- coordinates are `(row, col)`;
- `(0,0)` is the upper-left cell;
- movement is eight-connected;
- orthogonal movement costs 1;
- diagonal movement costs square root of 2;
- destination cells must be in bounds and unblocked;
- start and goal are never blocked;
- diagonal corner cutting is allowed;

- all classical neighbors come from `GridEnvironment.get_neighbors()`.

The first environment used seeded random obstacle generation. Dijkstra was implemented with a binary heap and reported the path, total cost, and number of visited nodes. Unit tests covered reachability, optimal cost, blocked cells, and basic environment behavior.

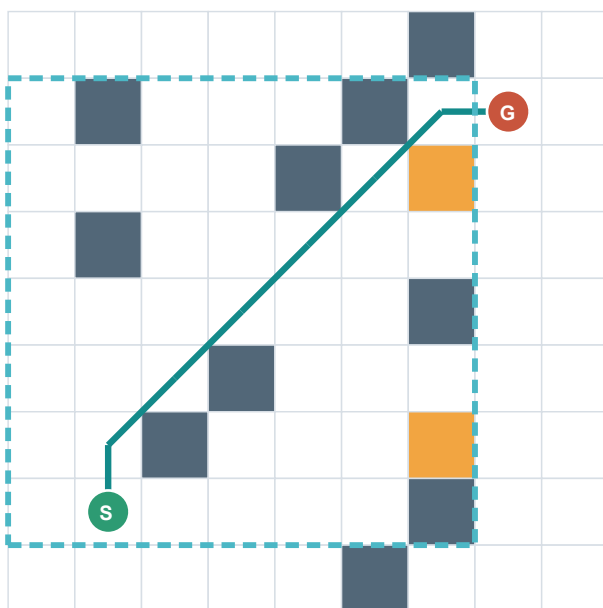
Early learning approaches

Before the current Double DQN+HER system, the repository included a tabular Q-learning route. The tabular state was simply the current grid cell. It used the same eight actions, epsilon-greedy exploration, a learning rate of 0.2, discount 0.95, and a shaped reward combining a step penalty, movement cost, goal reward, invalid-move penalty, and Euclidean progress. This implementation was useful as a transparent learning baseline on one fixed static grid, but it does not scale naturally to large or partially observed environments.

The deep-learning track moved to a goal-conditioned Gymnasium environment and Stable-Baselines3 DQN. The local observation window and a neural function approximator allowed the same model structure to act from many grid positions. Hindsight Experience Replay was added because goal-reaching rewards are sparse: a failed episode can still teach the agent how to reach states it actually visited by relabeling those achieved states as goals.

The current routing world

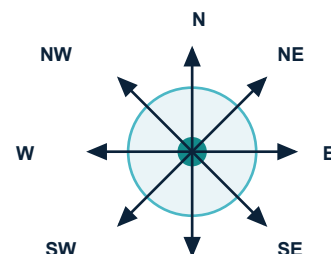
Shared grid world, local observation, and eight actions



Gray: static block Orange: changing cell Cyan box: 7x7 local observation

Cost contract

Straight 1, diagonal $\sqrt{2}$, plus any non-negative destination penalty.



Move-then-observe

Choose on current state; move; advance dynamics; expose the change before the next decision.

Figure 3. The same movement geometry and information timing govern classical planning and RL evaluation.

Grid, graph, and coordinates

The operating area is an $N \times N$ matrix. A free cell is a graph node. (row, col) coordinates are used consistently, with row increasing downward and column increasing to the right. The current benchmark includes grid sizes 15, 30, 50, and 100.

Each state can have up to eight outgoing moves:

Action	Delta	Base cost
North	$(-1, 0)$	1
South	$(1, 0)$	1
West	$(0, -1)$	1
East	$(0, 1)$	1
Northwest	$(-1, -1)$	$\text{sqrt}(2)$
Northeast	$(-1, 1)$	$\text{sqrt}(2)$
Southwest	$(1, -1)$	$\text{sqrt}(2)$
Southeast	$(1, 1)$	$\text{sqrt}(2)$

Corner cutting remains allowed: a diagonal move is legal when its destination cell is free, even if one of the orthogonally adjacent cells is blocked. This is a modeling choice, not a universal UAV rule, and it is applied equally to every method.

For classical planning, the cost of entering a neighbor is the geometric base cost plus any non-negative traversal penalty assigned to that destination. This supports controlled wind or energy maps without changing the graph interface.

Static obstacles and exact scenario injection

The earliest environments generated obstacles from a seed. That sounds reproducible, but the classical environment used Python's `random` module while the RL environment used NumPy's generator. Equal numeric seeds therefore did not guarantee equal blocked cells.

The first fairness repair directly copied the classical blocked grid into the RL environment for evaluation. The expanded study goes further: every benchmark record persists the exact blocked-cell list, start and goal, dynamics, penalties, no-fly configuration, noise level, and dynamics seed. Each method reconstructs the scenario from this record. Randomness is used to create the manifest, not to independently recreate a supposedly identical world inside each algorithm.

Three dynamic-obstacle families

The current environment implements three kinds of change:

1. A fixed-position obstacle toggles between blocked and passable on a fixed period.
2. A stochastic obstacle toggles independently with a seeded per-step probability.
3. A moving obstacle advances around a persisted cyclic path on a fixed period.

The original Week 3 benchmark used three fixed cells: $(4, 4)$, $(8, 8)$, and $(12, 11)$, each with period 5 and specified initial states. These positions were chosen on or near an important route so changes would matter. A single `default_dynamic_obstacles()` function became the source of truth for both the classical and RL sides.

The information-timing contract

The corrected contract is `post_move_observed`:

1. The method observes the current state.
2. It selects and executes one move under that state.
3. If the move did not end the episode, environmental dynamics advance.
4. The changed graph or grid is visible before the next decision.

No method reads future toggle times. A terminal move does not create an event that no method can observe or react to. If a cell becomes blocked while the UAV occupies it, the legacy benchmark permits the UAV to leave but not re-enter. That avoids an instantaneous, unavoidable collision, but it is only one physical interpretation and remains a stated limitation.

An earlier pre-move/hidden-change behavior was quarantined because it could make a policy act without seeing a change while a planner was evaluated under a different reaction opportunity. Checkpoints produced under that earlier contract are stored under `superseded_pre_observation_fix` and are inadmissible for the final study.

No-fly zones, energy, and sensor noise

No-fly handling evolved. It was included in the initial ambition, removed from the Week 3 comparison to reduce mismatch, and reintroduced in the expanded benchmark as an independently controlled factor.

- In hard mode, no-fly cells are removed from the traversable graph.
- In penalty mode, they remain traversable but add a configured cost.
- Traversal penalties represent spatial energy demand or wind exposure.
- Sensor noise randomly alters perceived cell values in the RL observation under a seeded probability.

These are abstractions. A traversal penalty is not an aerodynamic model, and a bit-flipped occupancy value is not a full sensor stack. Their purpose is to test sensitivity one factor at a time.

What the learning agent sees and does

Goal-conditioned observation

The default local observation contains 61 values:

- normalized UAV row and column: 2;
- normalized row and column displacement to the goal: 2;
- a flattened 7x7 local grid window: 49;
- a one-hot encoding of the previous action: 8.

The local window is centered on the UAV. Cells outside the map are encoded as obstacles, consistent with the movement boundary. A full-observation ablation replaces the 49-cell window with the entire 15x15 grid, increasing the input dimension and losing size invariance.

Stable-Baselines3 HER requires goal fields as well as the policy observation. `achieved_goal` stores the current and previous positions, while `desired_goal` stores the target position twice. Keeping the previous position allows the HER reward function to reproduce the potential difference for a relabelled transition.

The addition of relative displacement changed the policy observation from 53 to 61 values. The action ordering also changed during development. Both were correctly treated as breaking changes: older model weights could no longer be assumed compatible and had to be retrained.

Action and termination

The action space is the same eight movement directions used by the graph. Attempting an invalid destination is a collision and terminates the episode. Reaching the goal terminates successfully. An episode that exceeds its step budget is truncated. The default 15x15 budget is 225 steps.

The system distinguishes a crash, a timeout, and success in its information dictionary. That distinction was important in debugging: several early generalization failures were not collisions but repeated two-cell oscillations that exhausted the step budget.

Reward

The current sparse reward is:

Event	Sparse reward
Goal reached	+1

Event	Sparse reward
Collision or invalid move	-1
Ordinary step	-0.1

An ordinary move can also subtract an energy or penalized no-fly cost. Potential-based shaping then adds:

$$F(s,s') = \gamma \Phi(s') - \Phi(s), \gamma = 0.99$$

The default potential is normalized negative octile distance:

$$\Phi(s) = -d_{\text{octile}}(s, g) / d_{\text{max}}$$

Octile distance matches the eight-connected geometry:

$$d_{\text{octile}} = \min(\Delta r, \Delta c) \sqrt{2} + |\Delta r - \Delta c|$$

The step penalty discourages endless motion. Potential shaping gives a denser learning signal while preserving the intended policy ordering under its assumptions.

Why the potential changed

An earlier version used obstacle-aware shortest-path distance. In a static fixed grid, this is attractive because it rewards true progress around walls. It also created a major computational problem: HER asks the environment to recompute rewards for many relabelled goal pairs, so running a live shortest path for each sample made a 300,000-step run prohibitively slow. The project implemented an all-pairs distance table for fixed static grids, reducing those queries to constant-time lookups and avoiding an estimated roughly 13-hour run.

Dynamic HER exposed a second, more fundamental problem. A replayed transition was collected under one obstacle state. If its reward is later recomputed using the environment's current obstacle state, the reward no longer corresponds to the collected transition. The current protocol therefore uses obstacle-state-independent octile potential for dynamic HER. The old shortest-path potential remains available only for a compatible static experiment and is explicitly rejected for dynamic configurations.

Goal-conditioned Double DQN + HER learning loop

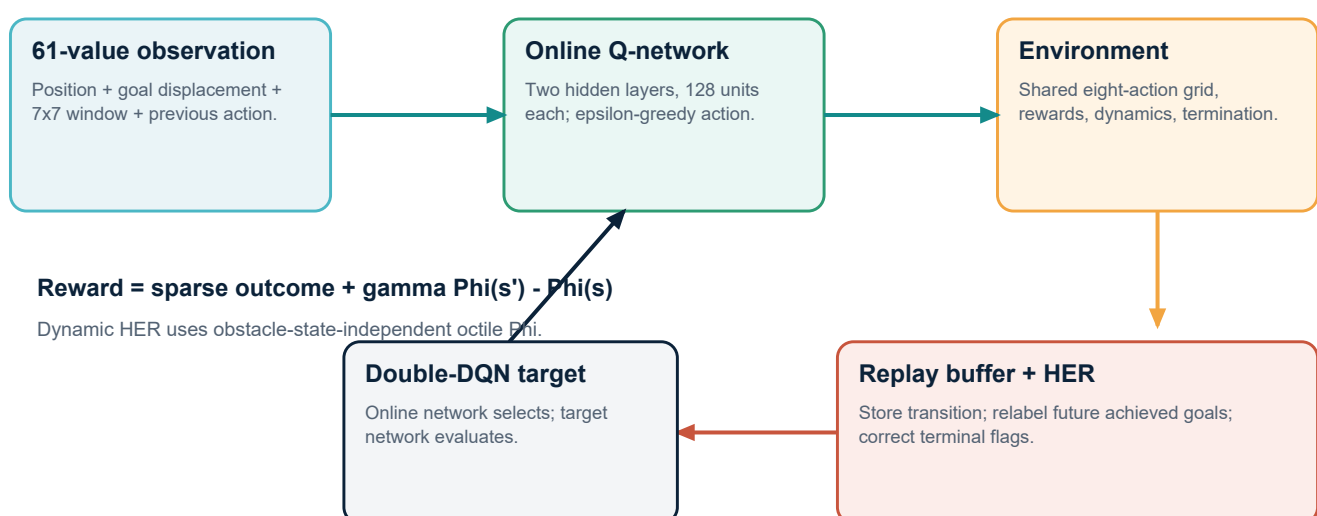


Figure 4. Correct reward and terminal semantics are required for HER relabeling to produce valid learning targets.

Algorithms considered and implemented

Dijkstra

Dijkstra expands the lowest known path-cost node until the goal is reached. With non-negative edges it returns an optimal route on the currently observed graph. In the dynamic runner, it calculates an initial route and performs a full new search after every observed change. With a binary heap its general complexity is:

$$O((V + E) \log V)$$

In an eight-connected grid, the number of edges grows with the number of free cells, so the cost is commonly described as approximately $O(V \log V)$. Dijkstra is not a weak or incorrect baseline; it is a deliberately simple full-replanning reference.

A*

A* uses the same graph and costs but prioritizes a node by accumulated cost plus an admissible estimate of remaining cost. The project uses octile distance, which is admissible for its eight-connected movement costs.

Because A* and Dijkstra share the same runner and change triggers, their difference isolates heuristic pruning. The completed pilot already shows why this matters: A* found equal-cost routes with substantially less search work than Dijkstra on the 15x15 dynamic scenarios.

D* Lite

D* Lite is the stronger dynamic classical reference. Instead of discarding all search work when edges change, it maintains *g* and one-step lookahead *rhs* values and repairs affected vertices. The implementation receives exactly the same changed-cell notifications as the full replanners. Its node-expansion count includes the initial computation and all repairs.

D* Lite prevents the final paper from presenting naive Dijkstra as the only classical response to change. A learned policy's deployment-time claim is interesting only if it is compared with both heuristic full replanning and an incremental planner.

Tabular Q-learning

The early tabular implementation stores one action-value vector for every visited cell. Its update is:

$$Q(s,a) \leftarrow Q(s,a) + \alpha [r + \gamma \max_{a'} Q(s',a') - Q(s,a)]$$

It is interpretable and useful for validating the action contract, but a cell index alone does not describe local obstacles, goal displacement, or a dynamic world. It therefore became a developmental baseline rather than the main method.

DQN and Double DQN

DQN replaces the table with a neural network. The current policy is a two-layer multilayer perceptron with 128 units in each layer. It uses a replay buffer, target network, epsilon-greedy exploration, minibatches, and gradient updates.

Standard DQN uses a maximization over target-network estimates. Double DQN separates action selection from evaluation: the online network chooses the next action, and the target network evaluates that action. This is intended to reduce overestimation bias. The project's `DoubleDQN` subclass implements this distinction directly and uses smooth L1 loss plus gradient clipping.

Hindsight Experience Replay

HER creates additional goal-conditioned learning examples from trajectories that did not reach their original goal. The configured replay buffer samples four future goals per transition. This is especially useful when a +1 goal reward would otherwise be rare.

The project discovered that relabelled transitions could retain an incorrect non-terminal flag when the relabelled achieved goal actually matched the desired goal. `SafeHerReplayBuffer` checks the relabelled positions and forces `done=1` for goal-reaching samples. Without that correction, bootstrapping can continue beyond an artificial terminal state and inflate value estimates.

How the research evolved

Eleven days of rapid methodological expansion



Git records stop at f57ae15; the expanded implementation and cloud workflow were still uncommitted at the cutoff.

Figure 5. The chronology explains why historical results and corrected v2 evidence must remain separate.

16 July: structure, grid, and Dijkstra

The first commit created the module structure. The same day, the project added the seeded grid environment and Dijkstra baseline. This established the shared coordinate and movement language on which the remaining work depended.

18-19 July: deep RL diagnosis and stabilization

The deep-learning work rapidly accumulated diagnostic scripts, training logs, trajectory traces, and toy experiments. This was not a clean linear training story. Reward definitions, observation composition, terminal behavior, and compute cost all had to be examined.

Key outcomes included:

- potential-based shaping;
- an all-pairs static distance table for efficient HER reward computation;
- a Colab-oriented training route;
- the HER terminal-flag correction;
- a -0.1 ordinary-step penalty to discourage loops and value inflation;

- no-fly density set to zero for the then-current controlled experiment;
- a 300,000-step Double DQN+HER checkpoint;
- a tabular Q-learning implementation;
- an unseen-seed diagnostic with 47 successes in 50 episodes.

The diagnostic record describes two different failure modes. One involved unstable or inflated action values during reward/terminal debugging. The other appeared on an unseen layout as a stable policy oscillation between two cells, causing timeouts rather than collisions. Treating those as distinct matters: one is a learning-target problem; the other is a generalization/representation problem.

21-22 July: dynamic obstacles and paired evaluation

Week 3 introduced pre-positioned toggling obstacles, a naive Dijkstra replanner, and a shared `DynamicObstacle` configuration. The RL model was fine-tuned through mild and full dynamic phases. Fifty shared scenario IDs were used for an early paired dynamic comparison, followed by Wilcoxon testing.

This early result was scientifically valuable because it contradicted an easy success narrative. Dijkstra remained more reliable and path-optimal. The learned policy adapted in most cases, but it did not win.

23-25 July: paper construction and stronger baseline

The project converted the early experiment into methodology, results, discussion, limitations, figures, and a LaTeX paper. It added static results, paper labels, author information, overflow fixes, and an A* revision. The original paper was archived as v1, while v2 became the current expanded manuscript line.

The v1/v2 split is important. It preserves the historical paper without allowing its conclusions to masquerade as the final expanded study.

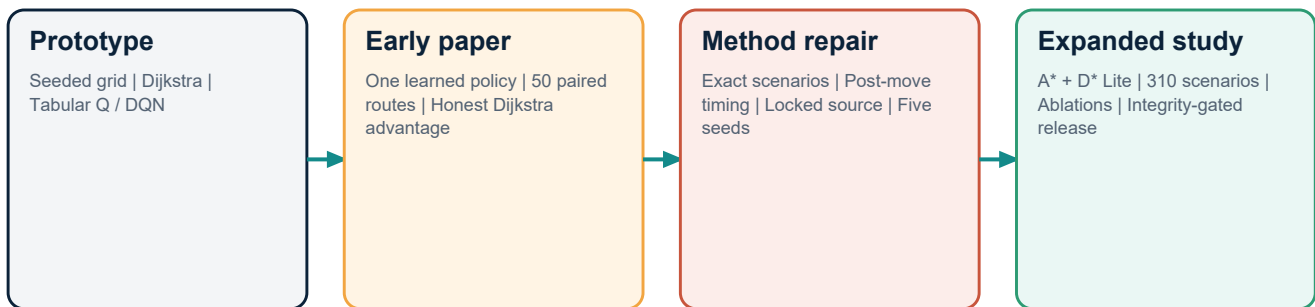
26-27 July: research redesign, corrected training, and cloud migration

The improvement plan turned identified weaknesses into ten workstreams: reproducibility, timing rigor, multiple RL seeds, generalization, scaling, stronger baselines, realistic factors, controlled ablations, adaptability, and paper/artifact release.

A 310-scenario benchmark, D* Lite, repeated timing, source manifests, adaptability analysis, statistical scripts, integrity checks, generated paper fragments, and a one-command pipeline were implemented. Training was restarted under the corrected `post_move_observed` contract. Five full-method seeds completed and were provenance-verified. The vanilla-DQN ablation began.

As the long sequential queue occupied the local machine, the computation was migrated to a free Google Colab T4 runtime. A notebook mounted Drive, restored the exact locked bundle, benchmarked the runtime, and ran a seed-resumable worker that synchronized completed artifacts to Drive. The local Python processes were stopped.

From a two-method demonstration to an auditable study



Core change: the project shifted from asking whether RL can route to asking exactly when, why, and with what uncertainty it is competitive.

Figure 6. Each expansion was motivated by a concrete weakness in the earlier evidence.

Problems discovered, fixes made, and lessons learned

Equal seeds were not equal environments

Problem: the classical and RL environments used different random generators. The same numeric seed did not guarantee the same obstacle layout.

Fix: first inject the exact classical blocked grid into the RL evaluator; then persist every expanded scenario and reconstruct it exactly.

Lesson: fairness must be expressed as shared data, not assumed from matching seed labels.

Dynamic obstacles could silently drift

Problem: separately configured obstacle positions or timing could make one method solve a different dynamic problem.

Fix: define shared obstacle dataclasses and one default fixed-obstacle list; later persist all dynamics per scenario.

Lesson: every changing object is part of the experimental input and must be versioned.

Changes occurred at an ambiguous time

Problem: applying dynamics before one method's move but after another's move changes what each method could know. Hidden pre-move changes can produce an unfair collision.

Fix: adopt `post_move_observed` for all methods, quarantine earlier research checkpoints, and retrain the full method.

Lesson: information availability is as important as map geometry in a dynamic benchmark.

Action and observation changes invalidated models

Problem: the action ordering changed, and relative goal displacement expanded the policy vector from 53 to 61 values. Old weights no longer represented the same function.

Fix: document the breaking change and retrain instead of attempting silent checkpoint reuse.

Lesson: a model checkpoint is meaningful only with its exact observation and action schema.

HER terminal flags were wrong after relabeling

Problem: a relabelled goal-reaching transition could continue bootstrapping as if it were non-terminal.

Fix: `SafeHerReplayBuffer` explicitly marks spatial goal matches as terminal.

Lesson: an algorithm name such as HER does not guarantee correct integration; goal, reward, and terminal semantics must agree.

Reward design encouraged pathological value behavior

Problem: sparse positive outcomes and incorrect continuation could inflate Q-values or fail to penalize endless movement sufficiently.

Fix: use a -0.1 step cost, collision penalty, goal reward, corrected terminal flags, potential shaping, and diagnostic trajectories.

Lesson: reward curves are not enough. Inspecting trajectories, timeouts, and action values can reveal qualitatively different failures.

Obstacle-aware HER shaping was computationally and semantically unsafe

Problem: live shortest-path calls inside HER reward computation were extremely slow. Under dynamics, recomputing with the current obstacle state could also assign the wrong reward to a replayed transition.

Fix: precompute all-pairs distances for compatible fixed static grids and use obstacle-independent octile potential for dynamic HER.

Lesson: reward recomputation must depend only on information stored with the transition or guaranteed invariant.

The first dynamic comparison was fair only by simplifying the world

Problem: static obstacles differed across environment implementations.

Fix: Week 3 set static density to zero and compared only three shared dynamic cells.

Trade-off: this repaired immediate fairness but made the benchmark easier and less representative.

Later fix: the expanded manifest persists exact static blocked cells and reintroduces density, no-fly, energy, stochastic, moving, and noise factors in controlled splits.

Lesson: simplifying a benchmark can be a valid temporary control, but the resulting limitation must remain visible.

One policy seed did not measure training uncertainty

Problem: many test routes from one trained network measure scenario variation, not variation in learning.

Fix: train policy seeds 11, 22, 33, 44, and 55 while holding layout seed 42 fixed.

Lesson: independent training seeds are the uncertainty units for RL.

Dijkstra alone was an incomplete classical comparison

Problem: repeated full Dijkstra search can make RL appear favorable while ignoring heuristic and incremental replanning.

Fix: add A* and D* Lite under the same graph, triggers, timing, and scenario contract.

Lesson: a baseline should represent credible alternatives, not merely the easiest algorithm to beat.

Millisecond timings needed a protocol

Problem: one timing number is sensitive to warm-up, hardware, implementation, and the boundary of what is timed.

Fix: repeat each route ten times; use `time.perf_counter()` only around planner or policy decision calls; retain raw repetitions; report route-level and per-decision distributions; attach machine/package metadata.

Lesson: timing is an experimental measurement, not an inherent property of an algorithm label.

Long training threatened reproducibility and local usability

Problem: a sequential 17.5-million-step training matrix can be interrupted, occupy the local computer, or accidentally mix source versions.

Fix: stage-level checkpoints, half-stage recovery checkpoints, per-seed metadata, a locked seven-file source digest, resumable queues, superseded artifact quarantine, and Drive-backed Colab execution.

Lesson: compute orchestration is part of the research method when experiments take longer than one interactive session.

The expanded research design

Research questions

The current manuscript asks four broad questions:

1. How do RL, naive search, heuristic search, and incremental search compare in reliability, path quality, and route-level decision cost?
2. How much does learned performance vary across independent training seeds?
3. How do methods respond to distribution shift, realistic cost factors, changing dynamics, and increasing grid size?
4. Which components of the learned method account for its performance?

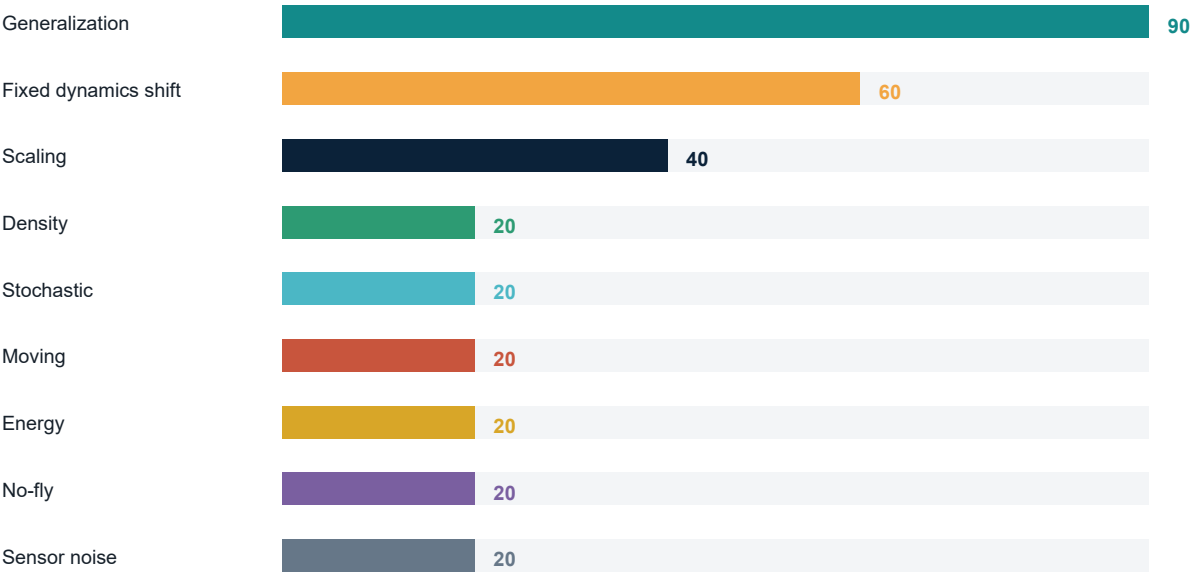
These questions are intentionally harder than the original "Dijkstra versus RL" question. They make a negative or mixed result useful because they locate the conditions under which each method succeeds.

The 310-scenario benchmark

The expanded manifest contains 310 persisted scenarios across 21 named splits. Of these, 280 use 15x15 grids, while 10 each use 30x30, 50x50, and 100x100 grids.

Expanded benchmark composition

310 persisted scenarios across 21 named splits



Grid sizes: 280 scenarios at 15x15; 10 each at 30x30, 50x50, and 100x100.

Figure 7. Scenario identity is persisted once and reconstructed identically for every method.

Family	Split	Scenarios	Purpose
Generalization	seen layout, unseen pairs	30	Held-out endpoints on the training layout
Generalization	unseen layout, same density	30	New geometry at density 0.20
Generalization	denser unseen layout	30	New geometry at density 0.30
Dynamics	new toggle locations	30	Spatial shift in fixed dynamics
Dynamics	changed toggle periods	30	Temporal shift in known locations
Scaling	15, 30, 50, 100	40	Computation and success versus grid size
Density	0.10 and 0.40	20	Low/high clutter
Stochastic	probability 0.05 and 0.20	20	Seeded unpredictable toggles
Moving	period 2 and 4	20	Cyclic moving obstacles
Energy	penalty 0.25 and 1.0	20	Spatial traversal-cost sensitivity
No-fly	hard and penalized	20	Regulatory constraint semantics
Observation	noise 0.05 and 0.10	20	Perception robustness

The manifest file SHA-256 at the cutoff was 296196e774e5aa7f45b988f0321355a85f4dcbe21d1c727842cc818ebde0e0b5. Every final raw result row must carry the matching manifest digest.

Training matrix

Each standard seed follows:

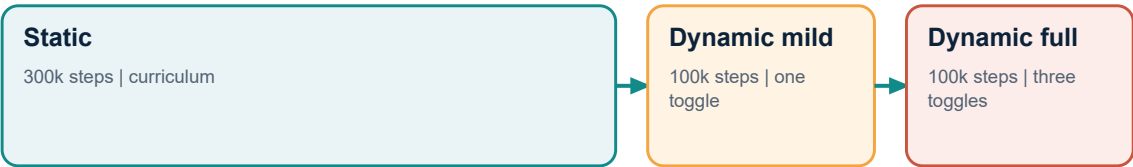
1. 300,000 static steps at obstacle density 0.20 with curriculum enabled when allowed by the variant.
2. 100,000 dynamic-mild steps with one fixed obstacle at (8,8), period 10.
3. 100,000 dynamic-full steps with the three shared fixed-toggle obstacles.

The full method uses Double DQN, HER, octile potential shaping, curriculum, local observation, and dynamic fine-tuning. Six controlled variants change a named factor:

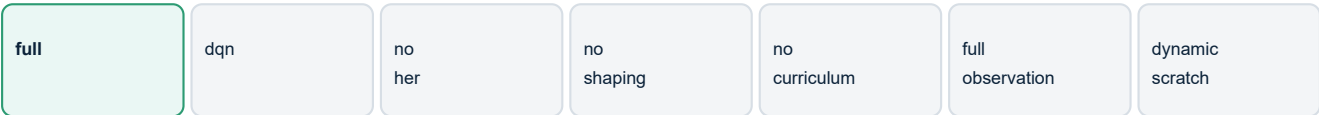
Variant	Controlled change	Steps per seed
full	Reference configuration	500,000
dqn	Standard DQN instead of Double DQN	500,000
no_her	HER disabled	500,000
no_shaping	Potential shaping disabled	500,000
no_curriculum	Start-distance curriculum disabled	500,000
full_observation	Complete 15x15 grid instead of 7x7 window	500,000
dynamic_from_scratch	Full dynamic condition only; no curriculum transfer	500,000

With five policy seeds, the configured total is 35 seed-variant trainings and 17.5 million environment steps. At the cutoff, nine complete admissible seed-variant metadata records represented 4.5 million verified steps. The remaining 13 million configured steps were not all unstarted - `dqn:seed055` had partial recovery artifacts - but they were not yet represented by complete admissible metadata and therefore remain unfinished evidence.

Five-seed curriculum and controlled ablation matrix



Policy seeds: 11, 22, 33, 44, 55 | layout seed: 42 | 500,000 steps per seed



Configured total: 35 seed-variant trainings = 17.5 million steps

Verified at cutoff: 5 full + 4 DQN seed completions = 4.5 million steps with complete metadata.

Figure 8. The dynamic-from-scratch variant spends its equal 500k budget entirely in the full dynamic condition.

Hyperparameters

Parameter	Value
Learning rate	0.001
Replay buffer	100,000
Learning starts	5,000
Batch size	256
Discount gamma	0.99
Train frequency	every 2 steps
Gradient steps	1
Target update interval	1,000
Exploration fraction	0.5
Initial/final epsilon	1.0 / 0.05
Network	128, 128
HER future goals	4

The layout seed remains 42 while policy seeds vary. This isolates optimization variation from layout variation during training. Generalization is then tested by the persisted benchmark rather than by silently changing the training map.

Fair timing

For a classical route, the timed quantity is the cumulative duration of all planner calls: the initial plan plus every repair or replan. For RL, it is the sum of policy `predict()` durations across the episode. Environment creation, stepping, metric calculation, plotting, and serialization are outside the decision timer.

Every method/scenario pair is repeated ten times for timing. Outcome and route metrics are deterministic under a fixed scenario and policy; the repetitions measure runtime variation. The final timing summary first collapses repetitions within a seed/scenario pair, preventing pseudoreplication.

Route-level timing is useful because it measures total decision computation needed to complete the route. Per-decision timing is also retained because a planner may make a few expensive decisions while a policy makes many small ones.

Statistical plan

For RL, seed-level estimates are the independent units. The mean and 95 percent Student-t confidence interval are calculated across the five trained seeds. Classical uncertainty uses scenarios as independent units where appropriate.

Paired success differences use exact McNemar tests. Paired path cost and route computation use two-sided Wilcoxon signed-rank tests. Rank-biserial effect sizes accompany the Wilcoxon results. Holm correction controls the familywise error rate across the reported comparisons.

Only jointly successful routes enter paired path-cost tests because a failed route has no valid comparable cost. All routes remain in success and computation analyses. This rule prevents failures from disappearing while avoiding invented costs.

Adaptability as an event-level outcome

The early project used "adaptability" mostly as a general description of whether a policy could continue. The expanded study defines it per change event.

Immediately before and after a visible change, the evaluator computes the optimal remaining cost. The difference is the optimal-cost shock:

$$\Delta C^* = C^*_{\text{after}} - C^*_{\text{before}}$$

Positive extra optimal cost is $\max(0, \text{delta})$. Recovery steps count decisions until the current optimal remaining cost returns to or below its pre-change value. Unrecovered events retain a missing recovery time instead of being discarded. The event also records eventual route success, planner repair time and expansions, or policy reaction latency.

For paired event tests, matching uses scenario ID, change step, and changed-cell signature. Event order alone is insufficient because two methods may experience a different number of later events after an earlier failure.

Integrity before interpretation

The final artifact gate is not a loose row-count check. It verifies:

- the exact expected seed x scenario x repetition Cartesian product;
- unique run IDs;
- the exact benchmark-manifest hash;
- rejection of smoke-test and legacy checkpoints;
- complete parent-route coverage for every adaptability event;
- correct seed/stage coverage;
- presence of final generated outputs.

The paper compiler refuses to publish a release PDF unless the integrity report passes and placeholder fragments have been replaced. This prevents a polished manuscript from outrunning its evidence.

The computational workflow

Local development and training

The project began as a local Python repository on Windows. A virtual environment isolated Gymnasium, Stable-Baselines3, PyTorch, NumPy, SciPy, NetworkX, Matplotlib, and pytest. Checkpoints were saved at the midpoint and end of each stage. Stage metadata records the source checkpoint, output checkpoint, executed steps, elapsed seconds, and steps per second.

Resumption happens at a stage boundary. A dynamic-mild stage requires the static final checkpoint; dynamic-full requires dynamic-mild. The dynamic-from-scratch variant creates a new model directly in its only requested stage.

The local logs reveal several generations of training:

- exploratory and diagnostic models;
- a five-seed run later superseded by the observation-timing correction;
- the corrected full five-seed run;
- the corrected sequential ablation queue.

Superseded artifacts are moved into named quarantine directories instead of being silently overwritten. That is essential because file existence alone does not establish methodological validity.

Provenance lock

Seven files define the training-relevant source boundary:

1. `configs/research_experiments.json`
2. `envs/grid_environment.py`
3. `experiments/train_multiseed.py`
4. `requirements.txt`
5. `rl_agent/double_dqn.py`

6. `rl_agent/safe_her_buffer.py`

7. `rl_agent/uav_env.py`

Their ordered aggregate SHA-256 is:

`00a4ff215b3d31f7be2a42d62f7467d19beacd3f3e78c8684efe2d233412f39d`

The provenance script recomputes every file hash and fails if the current source differs from the locked snapshot. Completed non-smoke metadata is then annotated with that snapshot. This protects the active experiment from a subtle but common error: changing a reward or transition rule halfway through a multi-seed campaign.

Why training moved to the cloud

The corrected full-method seeds each required roughly one to 1.34 hours on the local machine in the recorded run. Vanilla-DQN seeds ran more slowly in the available local environment. With 26 seed-variant completions still missing, the sequential queue would continue to consume the user's computer.

The user requested that remaining training and computation be removed from the local machine and performed on cloud servers without enabling billing. The selected runtime was the free Google Colab T4 available to the account.

At dispatch, a 20,000-step benchmark reported 82.09 seconds, or 243.64 steps per second, on CUDA. This is a runtime snapshot, not a guaranteed sustained rate. Free Colab sessions can disconnect or change hardware.

Drive-backed resumability

The cloud notebook:

1. mounts Google Drive;
2. locates the exact project bundle;
3. extracts it into the Colab runtime;
4. installs the required environment;
5. confirms the locked source snapshot;
6. benchmarks the current runtime;
7. launches selected variants and seeds;
8. synchronizes each completed seed to Drive;
9. writes a status JSON and log in the backup root.

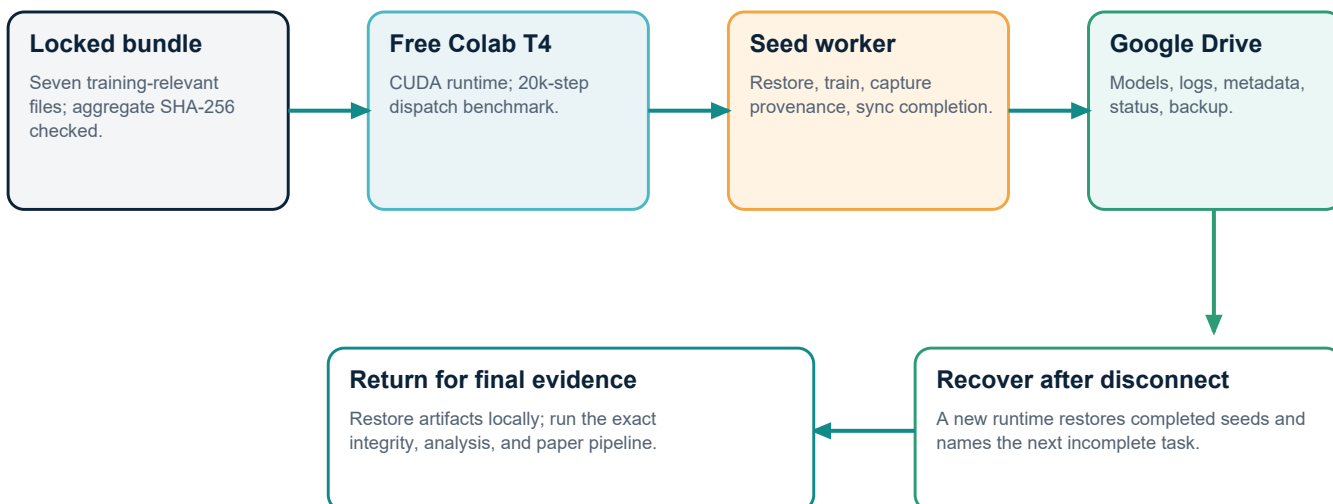
Before training a seed, the worker restores any Drive backup. After successful training and provenance capture, it copies the model directory, run logs, training metadata, and source snapshot back to Drive. A time-budget check stops only between seeds, leaving the next task named in status.

The recorded Drive locations include:

- `MyDrive/UAV Dynamic Routing/uav_cloud_bundle.zip`
- `MyDrive/UAV Dynamic Routing/cloud_completed_artifacts.zip`
- `MyDrive/UAV Dynamic Routing/research-backup`
- `research-backup/colab_worker_status.json`
- `research-backup/colab_unified_worker.log`

The local dispatch record states that remaining training and artifact computation belongs to Colab and that the local queue must not be restarted. At the cutoff, no local Python processes were active.

Drive-backed free-Colab execution



Dispatch snapshot: 243.64 steps/s on a free T4; local Python processes stopped.

Figure 9. Drive improves recovery, but final validity still depends on provenance, stage coverage, and artifact integrity.

What Drive backup does and does not guarantee

Drive protects completed seed artifacts from a Colab runtime reset. It does not guarantee that an interrupted in-memory stage can resume from the exact last gradient update. Recovery granularity depends on the saved midpoint and final checkpoints. It also does not make an artifact valid by itself: restored files must still match the locked digest, expected stages, non-smoke flag, and final integrity gate.

Results available at the cutoff

Evidence tiers must not be blended

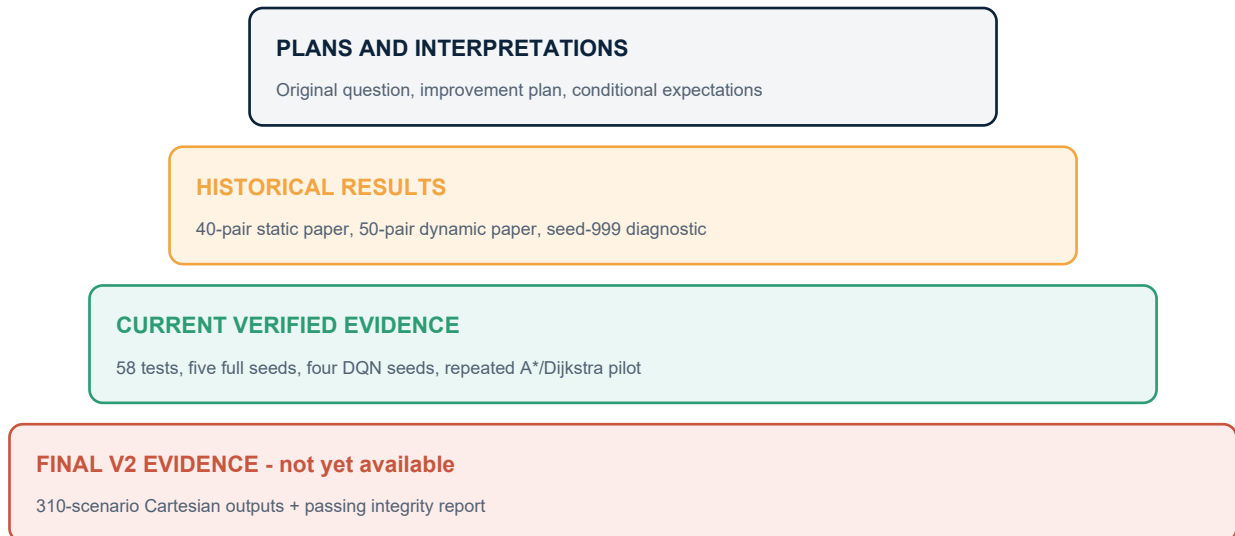


Figure 10. Earlier measurements explain the project history but cannot substitute for the corrected expanded-study outputs.

Historical static comparison

The archived v1 paper reports 40 paired routes on one 15x15 grid at obstacle density 0.20. The exact classical blocked grid was copied into the DQN evaluation environment. Dijkstra succeeded in 40/40 routes; the single learned policy succeeded in 39/40. Among successful learned routes, the mean path-cost gap above Dijkstra was 0.9248 and the maximum was 3.6569. Fourteen learned routes exactly matched Dijkstra, and 25 were longer. The one failure was a 225-step timeout rather than a collision.

The archived manuscript reports mean route-level computation of 0.78 ms for one Dijkstra query and 3.20 ms for accumulated DQN predictions on successful routes.

HISTORICAL RESULT. This experiment is useful evidence about the early system, but it uses one policy and the earlier research protocol. It is not a final v2 result.

Historical Week 3 dynamic comparison

The early dynamic benchmark used 50 shared start-goal pairs, no static obstacles, and three shared period-5 toggle cells. Dijkstra succeeded in all 50 routes. The single dynamically fine-tuned RL policy succeeded in 49.

On the 49 jointly successful routes:

- mean path-cost difference, RL minus Dijkstra: +1.369441;
- median difference: +0.828427;
- RL lower/equal/higher: 0 / 16 / 33;
- Wilcoxon statistic: 0;
- unadjusted p-value: 5.644724e-07.

The early summary reported mean Dijkstra decision computation of 4.627753 ms and RL computation of 5.334265 ms on the jointly successful set, with an unadjusted Wilcoxon p-value of 0.062865.

The correct interpretation was deliberately modest: the policy adapted and usually arrived, but Dijkstra was more reliable and produced shorter or equal routes. No significant compute advantage was demonstrated at 15x15.

HISTORICAL RESULT. These CSVs predate the final multi-seed, expanded, integrity-gated evaluation and must not be pooled with it. The observation timing correction later forced research checkpoint quarantine and retraining.

Historical unseen-layout diagnostic

The training diagnostic report records 47 successes in 50 episodes on a seed-999 static layout, with zero crashes and three timeouts. The failures were two-cell oscillations. This showed some transfer beyond the training layout and exposed a specific generalization failure.

HISTORICAL DIAGNOSTIC. It is not the current five-seed generalization study and should not be reported as a final robustness rate.

Verified repeated classical pilot

The strongest complete result under the current repeated-timing machinery is the Dijkstra/A* pilot. Each planner ran 50 dynamic scenarios with ten repetitions, producing 500 route runs per planner.

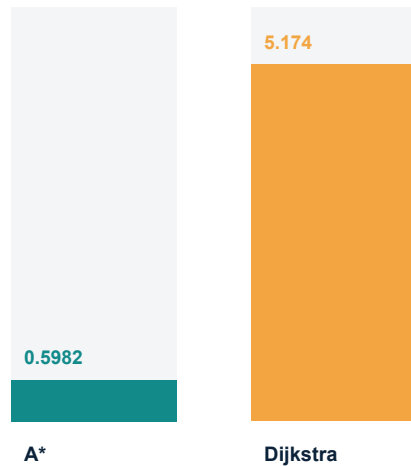
Planner	Runs	Success	Mean path cost	Mean route time	Median route time	Mean expansions
A*	500	1.000	10.5637	0.5982 ms	0.4505 ms	32.6
Dijkstra	500	1.000	10.5637	5.1742 ms	4.4935 ms	251.14

The raw normal-approximation 95 percent timing half-widths were 0.0415 ms for A* and 0.2985 ms for Dijkstra. The equal mean path cost is expected when both optimal planners use the same graph and admissible heuristic. The expansion and runtime difference demonstrates the value of heuristic pruning in this implementation and hardware environment.

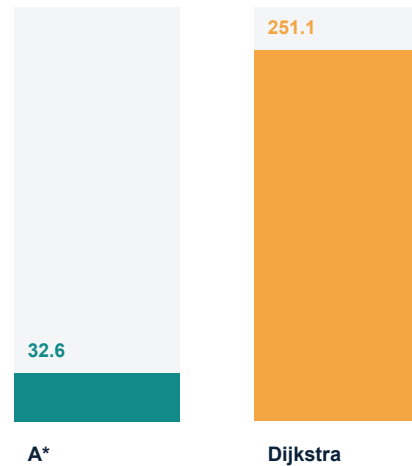
This pilot is not the final classical suite. It does not yet include the 310 expanded scenarios or D* Lite, and its raw timing confidence interval treats route repetitions differently from the final collapsed analysis. It is still a completed, auditable current artifact.

Repeated classical pilot: equal route cost, different search work

Mean route time (ms)



Mean node expansions



Both planners: 500/500 successes; mean path cost 10.5637.

Hardware-specific Python measurements; final suite will add D Lite and the full 310-scenario manifest.*

Figure 11. A demonstrates why the final RL claim must be tested against stronger classical methods than naive Dijkstra alone.*

Verified training completion

Five corrected full-method training metadata files exist, one for each policy seed. Every seed contains static, dynamic-mild, and dynamic-full stages, 500,000 executed steps, and the locked source digest.

Seed	Total steps	Elapsed hours	Aggregate steps/s
11	500,000	0.989	140.4
22	500,000	0.983	141.3
33	500,000	0.990	140.3
44	500,000	1.150	120.8
55	500,000	1.344	103.3

Four corrected vanilla-DQN seeds were complete:

Seed	Total steps	Elapsed hours	Aggregate steps/s
11	500,000	1.046	132.7
22	500,000	1.969	70.6
33	500,000	1.667	83.3
44	500,000	1.721	80.7

Training completion is not performance. These records prove that the planned interactions were executed under the locked code and show their compute throughput. They do not establish success, generalization, or superiority until the checkpoints are evaluated on the persisted suite.

Verified implementation checks

The local test command completed with 58 passing tests in 43.70 seconds. The only warning was that pytest could not create its cache directory due to local file permissions. The warning did not indicate a failed test.

Coverage includes:

- Dijkstra, A*, and D* Lite behavior;
- static and dynamic environment rules;
- fixed, stochastic, and moving obstacles;
- realism factors and no-fly handling;
- scenario loading and uniqueness;
- timing-boundary behavior;
- experiment metadata;
- adaptability calculations;
- artifact-integrity logic;
- Tectonic installer checks;
- RL configuration and dynamic timing.

Tests support implementation confidence. They do not replace experimental results.

Exact interim state

The local queue status at the cutoff said:

- state: `migrated_to_cloud`;
- active variant: `dqn`;
- complete seeds: 11, 22, 33, 44;
- pending seed: 55;
- local compute: stopped.

The cloud dispatch record said:

- provider: Google Colab;
- billing enabled: false;
- runtime: free T4 GPU;
- worker state: training;
- active task: `dqn:seed055`;
- remaining variant order: `dqn`, `no_her`, `no_shaping`, `no_curriculum`, `full_observation`, `dynamic_from_scratch`;
- local active Python processes at stop: 0.

Local recovery files showed completed static and dynamic-mild checkpoints and a partial dynamic-full checkpoint for DQN seed 55, but no complete admissible training metadata file existed. The documentary therefore labels the task in progress instead of inferring completion from a checkpoint filename.

The expanded generated LaTeX fragments were still placeholders. The existing `paper_latex_v2/main.pdf` explicitly identifies itself as non-current until the queue finishes and the integrity report passes. No passing final `integrity_report.json` was available.

What remains before the research is finished

Path from interim status to final release



Figure 12. Completion is defined by admissible artifacts and verified release outputs, not by the end of training alone.

1. Finish the ablation training matrix

Colab must finish DQN seed 55 and then train all five seeds for no-HER, no-shaping, no-curriculum, full-observation, and dynamic-from-scratch. Each completion must be synchronized to Drive and later restored locally with its models, progress logs, metadata, and matching source snapshot.

2. Reconcile provenance and stage coverage

Every restored metadata file must be non-smoke, use the locked digest, and contain exactly the expected stages. Superseded pre-observation-fix artifacts must remain excluded. Missing or mixed-source seeds must be retrained rather than patched into the final table.

3. Run the complete classical suite

Dijkstra, A*, and D* Lite must be evaluated on every expanded scenario with all repetitions. The result must preserve per-route computation, per-decision latency, node expansions, success, route cost, and change events.

4. Evaluate every learned variant

Each of the seven configurations must be evaluated for five seeds across all 310 scenarios and timing repetitions. Missing checkpoints should cause a hard failure; no legacy model may be substituted.

5. Pass the exact Cartesian integrity gate

The gate must confirm every expected combination, not merely approximately the right number of rows. Adaptability events must link to valid parent routes. Manifest hashes and run IDs must be exact and unique.

6. Generate seed-aware summaries and formal tests

The pipeline must produce research summaries, timing distributions, paired tests, effect sizes, Holm-adjusted p-values, adaptability summaries, and event-level tests. Failed routes and unrecovered events must remain visible.

7. Generate figures, tables, and evidence-bound prose

Generalization, scaling, realism, ablation, and adaptability plots must come from final raw data. Manuscript table fragments and abstract/results/conclusion sentences must be regenerated. Any unexpected result should change the narrative rather than be hidden.

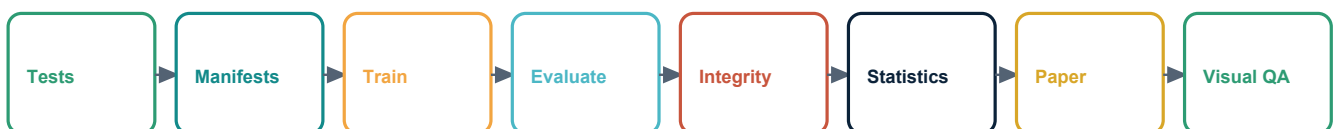
8. Compile and visually inspect the release paper

The pinned compiler must build the release PDF only after integrity passes. Every page must be rendered to images and inspected for overflow, clipping, unreadable labels, broken equations, bad floats, empty pages, and inconsistent formatting. The final build manifest must record the compiler, integrity hash, PDF hash, and page count.

9. Produce the definitive documentary

This documentary must then be updated. Historical results should remain as history, but the current-state and remaining-work chapters must be replaced by final measurements. Conditional contribution language must become either a supported conclusion or a documented rejected hypothesis.

Integrity-gated artifact pipeline



Release rule No passing integrity report or remaining placeholder -> no release PDF.

After visual corrections, the final gates must run again so the inspected PDF and verified evidence remain synchronized.

Figure 13. The paper compiler is downstream of evidence completeness and upstream of page-by-page visual inspection.

What the final results may establish

The following are interpretations and possible outcomes, not current findings.

If the learned policy maintains high success while route-level computation grows more slowly than classical search at 50x50 or 100x100, the project may demonstrate a real deployment-compute trade-off. If D* Lite remains both fast and reliable, the project may instead show that incremental search is the stronger choice under the tested graph dynamics.

If HER, shaping, or curriculum produces a consistent five-seed advantage, the ablation study will explain why the full method works. If an ablation matches the full method, the research should simplify its claimed contribution.

If local observation generalizes better across grid sizes or layouts, that would support representation invariance. If full observation performs better only at 15x15, the result would reveal a dimension-specific trade-off.

If performance collapses under unseen layouts, new obstacle positions, sensor noise, or denser clutter, the most important contribution may be a clear boundary on fixed-layout RL. A careful negative result is scientifically more useful than an unsupported success claim.

Under no outcome can the grid study by itself prove real-world UAV safety. At most, it can establish algorithmic behavior under a precisely defined routing abstraction and motivate later work in three-dimensional dynamics, safety buffers, localization uncertainty, embedded hardware, and multi-UAV settings.

Expected contribution, conditional on final evidence

Assuming the final artifacts pass integrity and support the manuscript's claims, the completed project will contribute:

1. A persisted and reconstructable dynamic-routing benchmark with 310 scenarios spanning endpoints, layouts, density, dynamics, scale, cost factors, no-fly semantics, and noise.
2. A controlled comparison of Dijkstra, A*, D* Lite, and multi-seed deep RL under one graph, movement, information, and timing contract.
3. A five-seed Double DQN+HER training protocol with controlled ablations of algorithm, replay, shaping, curriculum, observation scope, and dynamic training schedule.
4. An event-level definition of adaptability that preserves unrecovered events and measures both route consequences and reaction computation.
5. A reproducibility package with stable IDs, raw repetitions, environment metadata, locked training source, resumable checkpoints, exact integrity checks, generated manuscript evidence, and a pinned release build.
6. An honest empirical answer about where learned routing is competitive, not merely whether a single policy can complete a small dynamic grid.

If the remaining data do not support one or more of these points, the final paper must narrow the contribution accordingly.

Reproducing and auditing the project

Key entry points

Task	Command or path
Run tests	<code>python -m pytest -q</code>
Reproduce repeated classical pilot	<code>python scripts/reproduce_classical.py</code>
Validate expanded manifest	<code>python evaluation/validate_benchmark.py</code>
Train one variant	<code>python experiments/train_multiseed.py --variant <name></code>
Run full evidence pipeline from checkpoints	<code>python scripts/run_full_research.py</code>
Train and run everything on a clean checkout	<code>python scripts/run_full_research.py --train</code>
Capture/check provenance	<code>python scripts/capture_training_provenance.py</code>
Compile release paper after integrity	<code>python scripts/compile_paper.py</code>
Current protocol	<code>docs/EXPERIMENT_PROTOCOL.md</code>
Artifact map	<code>docs/ARTIFACT_MAP.md</code>
Interim execution status	<code>docs/RESEARCH_EXECUTION_STATUS.md</code>

One-command pipeline order

The full orchestrator performs:

1. tests;
2. pilot and expanded manifest generation;
3. benchmark validation;
4. optional training;
5. provenance capture;
6. expanded classical evaluation;
7. classical adaptability extraction;
8. per-variant multi-seed evaluation;
9. artifact integrity;
10. research summaries;
11. timing distributions;
12. route-level statistical tests;
13. adaptability analysis and event tests;
14. research plots;
15. paper tables;
16. evidence-bound paper narrative;
17. LaTeX source checks;
18. pinned compiler installation;
19. paper compilation;
20. final status generation.

An error stops the pipeline because subprocesses run with `check=True`. This is preferable to silently producing a partially current paper.

Recreating the definitive documentary

After the cloud worker completes:

1. restore all Drive-backed models, run logs, and metadata into their original project-relative locations;
2. inspect cloud status and confirm the worker completed every variant/seed;
3. run `scripts/capture_training_provenance.py`;
4. run `scripts/run_full_research.py` without `--train`;
5. confirm `evaluation/results/integrity_report.json` says passed;
6. verify every generated CSV, table, figure, narrative fragment, and release PDF timestamp follows the raw results;
7. visually inspect every release-paper page;
8. update this Markdown source with the final values and interpretations;
9. change the title page from interim to definitive;
10. regenerate this documentary PDF with its generator;
11. render and inspect every documentary page again;
12. update the evidence manifest, pending-claims file, cutoff time, hashes, commit, working-tree state, and final paper path.

Research limitations that remain even after completion

The final experiment can close evidence gaps inside the benchmark, but it cannot remove the benchmark's conceptual limits.

- The world is two-dimensional and discrete.
- Motion has no turn radius, acceleration, altitude, or vehicle dynamics.
- Corner cutting is permitted.
- Dynamic obstacles follow stylized fixed, stochastic, or cyclic rules.
- Energy and wind are traversal penalties, not physical forces.
- Sensor noise is an occupancy perturbation, not a calibrated perception model.
- There is one UAV, no communication network, and no multi-agent conflict.
- The occupied-cell toggle rule is a legacy escape convention.
- Training begins on one fixed static layout.
- The full-observation ablation is tied to 15x15 input dimensions.
- Timing results describe the recorded Python/PyTorch implementations and hardware, not universal algorithm constants.
- No learned policy has a formal safety or shortest-path guarantee.

These limitations should shape future work: continuous 3D simulation, vehicle kinematics, safety buffers, uncertainty-aware planning, randomized training distributions, embedded inference, incremental classical implementations optimized to the same degree as neural inference, and eventually controlled real-flight validation.

Chronological ledger

Date	Commit	Evidence of project evolution
2026-07-16	9da9099	Initial module structure
2026-07-16	c3bc6b2	15x15 grid and Dijkstra baseline
2026-07-18	26f2cb6	HER refactor, potential shaping, all-pairs distance optimization, Colab setup
2026-07-19	f41fc73	HER terminal fix, step penalty, cleaned 300k Double DQN+HER model
2026-07-19	8056833	Static tabular Q-learning
2026-07-19	588a4fe	Seed-999 47/50 diagnostic and timeout analysis
2026-07-19	f182275	Cleanup and rigorous multi-pair evaluation
2026-07-21	bc40465	Pre-positioned dynamics and naive Dijkstra replanning
2026-07-21	016a641	Shared DynamicObstacle configuration
2026-07-22	7d702d4	Dynamic curriculum retraining and shared 50-scenario RL evaluation
2026-07-22	02fbaba	Dynamic baseline evaluation and paired statistics
2026-07-22	6ef621d	Paper documentation
2026-07-23	27f8c76	LaTeX paper draft and Week 4 documentation
2026-07-24	multiple	Static integration, labels, limitations, citations, and layout fixes
2026-07-25	7360312	v2 A*, hyperparameters, remeasured timings, honest limitations
2026-07-25	15d7894	Archived v1 and current v2 split
2026-07-26	f57ae15	Evaluation fixes and paper materials; current Git HEAD
2026-07-26/27	working tree	Expanded benchmark, D* Lite, multi-seed/ablation pipeline, statistics, adaptability, provenance, integrity, cloud workflow

The final row is not represented by a committed hash at the cutoff. The companion evidence manifest therefore records both the HEAD and dirty-tree boundary.

Technical appendix

DQN target

For a sampled transition, standard Q-learning uses:

$$y = r + (1 - d) \gamma \max_{a'} Q_{\text{target}}(s', a')$$

where d is the terminal flag. Double DQN instead selects with the online network and evaluates with the target network:

$$a^* = \arg \max_{a'} Q_{\text{online}}(s', a') \mid y = r + (1 - d) \gamma Q_{\text{target}}(s', a^*)$$

The HER terminal correction matters because an incorrect $d=0$ adds a future value after the relabelled goal has already been reached.

Path cost

For route nodes $v_0 \dots v_T$, geometric and traversal cost is:

$$C = \sum_{t=0}^{T-1} [c_{\text{move}}(v_t, v_{t+1}) + p(v_{t+1})]$$

where c_{move} is 1 or square root of 2 and p is a non-negative destination penalty. A failed route has no valid success-path cost and is not assigned a fictional penalty for paired path-cost testing.

Success and timeout

Success means the route reaches its persisted goal. A collision terminates after an invalid move. A timeout means the route exceeded its maximum steps without reaching the goal or crashing. Reporting these separately distinguishes safety failure from inefficient looping.

Confidence intervals and independence

Ten repeated timings from one seed on one scenario are runtime repetitions, not ten independent learned policies. The final analysis collapses these before seed-level inference. With five RL seeds, the confidence interval reflects between-training variation, though five remains a small sample and should be interpreted accordingly.

Why local and full observation answer different questions

The 7x7 window tests whether a fixed-dimensional reactive representation can generalize across positions and potentially across grid sizes. Full observation tests whether access to the complete 15x15 occupancy state helps on that dimension. It is not directly portable to 30x30 or 100x100 without a different input layer, so scaling conclusions must keep this distinction.

Glossary

Term	Plain-language meaning
Ablation	A controlled variant that removes or changes one component to measure its contribution
A*	Shortest-path search guided by an admissible estimate of remaining cost
Checkpoint	Saved model state used for recovery or later evaluation
Dijkstra	Optimal non-negative-edge shortest-path algorithm without a goal heuristic
D* Lite	Incremental planner that repairs prior search information after graph changes
DQN	A neural network that estimates the value of each discrete action
Double DQN	DQN variant separating next-action selection from target evaluation
Distribution shift	Test conditions that differ from the training distribution
HER	Replay method that relabels achieved states as alternative goals
Integrity gate	Automated proof that expected raw artifacts are complete, unique, and traceable
Manifest	Persisted machine-readable definition of scenarios or environment metadata
McNemar test	Exact paired test for differences in binary outcomes such as success
Octile distance	Diagonal-aware distance for an eight-connected grid
Policy seed	Random seed controlling an independent RL training run
Potential shaping	Reward term based on change in a state potential
Provenance	Evidence connecting an artifact to exact code, configuration, data, and environment
Rank-biserial effect	Effect size associated with paired rank comparisons
Replanning	Computing or repairing a route after the graph changes
Wilcoxon signed-rank	Nonparametric test for paired numeric differences

Closing perspective

The project did not progress by simply training a larger model. It progressed by making the comparison harder to fool.

The first grid and Dijkstra implementation gave the team a common routing language. Early DQN+HER work exposed reward, terminal, compute, and generalization failures. The Week 3 comparison produced a result that favored the classical baseline and forced an honest paper. The improvement plan then converted that honesty into experimental structure: stronger baselines, multiple seeds, exact scenarios, distribution shifts, realistic factors, event-level adaptation, provenance, integrity, and reproducible artifact generation.

At this interim cutoff, the project has a verified full-method training set, a partly completed ablation campaign, a tested expanded implementation, and a cloud-backed path to completion. It does not yet have the final evidence needed to decide the central trade-offs. That unresolved state is not a defect in this documentary; it is the most accurate description of the research.

The definitive contribution will be determined by what the final data show. The project's strongest methodological commitment is that the story must follow those data.